

C# Interfaces — The Beginner's Guide

Understanding Contracts, Real-world Analogies, Rules & Abstract Class Comparisons

1. What is an Interface? (The 5-Second Explanation)

An **Interface** is a strict **contract** or **promise**. It tells a class: *"If you want to use my name, you **MUST** implement all the actions I define."*

An interface only lists **WHAT** a class must do, never **HOW** it does it. It contains no state (fields/data) and usually no working code.

💡 Real-World Analogy: The Wall Power Outlet

Think of an interface like a standard 2-prong electrical wall socket (`IPowerable`):

- The socket promises: *"If you fit here, I will give you electricity."*
- A **TV**, a **Lamp**, and a **Laptop Charger** all plug into the exact same socket.
- The socket doesn't care *how* the TV displays picture or *how* the Lamp makes light — it only cares that they all agree to plug in the same way!

2. Rules of Interfaces (What You Can & Cannot Do)

Naming Convention:

In C#, all interface names **MUST** start with a capital **'I'** (e.g., `IDamagable` , `IPrintable` , `IPayer`).

What You **MUST** Do (When Implementing an Interface):

- **Implement Every Member:** You must write the actual code for every method/property defined inside the interface.
- **Make Members Public:** Methods implementing interface rules must be marked `public` .

What Interfaces **CANNOT** Do:

- **Cannot create instances:** You can NEVER write `new IPowerable()` (Compile Error).
- **Cannot store instance fields/data:** You cannot store variables like `int age;` directly inside an interface.
- **No constructors:** Interfaces cannot have constructors (`public IPowerable() { }` is invalid).

★ The Superpower of Interfaces: Multiple Inheritance!

In C#, a class can only inherit from **ONE** base/abstract class. However, a class can implement **UNLIMITED** interfaces!

```
public class Doctor : Person, IWorkable, IPayer, IHealer { }
```

3. Abstract Class vs Interface (When to use which?)

Feature	Abstract Class (What something IS)	Interface (What something CAN DO)
Core Purpose	Shared identity & base code setup.	A strict contract/capability across unrelated types.
Inheritance Limit	Single (Can only inherit 1 class).	Multiple (Can implement many interfaces).
Fields & Data	Can have instance variables & fields.	Cannot have instance variables/fields.
Code Implementation	Can mix full methods with abstract ones.	Mostly signatures only (pure contract).
Analogy	A Doctor IS A Person .	A Doctor CAN DO IDriving .

4. Easy-to-Understand Code Example

```
// 1. DEFINE THE CONTRACT (Notice the 'I' prefix)
public interface IPrintable
{
    // No body! Just stating that anything printable MUST have a Print method.
    void Print();
}

// 2. CLASS 1 IMPLEMENTS THE INTERFACE
public class PatientReport : IPrintable
{
    public string PatientName { get; set; }

    public void Print()
    {
        Console.WriteLine($"Printing medical record for {PatientName}...");
    }
}

// 3. UNRELATED CLASS 2 IMPLEMENTS THE SAME INTERFACE
public class Invoice : IPrintable
{
    public decimal TotalAmount { get; set; }

    public void Print()
    {
        Console.WriteLine($"Printing billing invoice of ${TotalAmount}...");
    }
}

// 4. POLYMORPHISM IN ACTION!
public class PrinterService
{
    public void PrintDocument(IPrintable doc)
    {
        // Works for PatientReport, Invoice, or ANY class implementing IPrintable!
        doc.Print();
    }
}
```